

Setting Up a Web API Project: Quick Reference

Introduction

This guide will help you set up a basic Web API using **ASP.NET Core**. We'll take it step-by-step, starting from installing the right tools to writing and testing your first API endpoint. This is meant for beginners who are just getting started with web development and only have **Visual Studio Code (VS Code)** installed.

What You'll Need Before You Begin

To follow this guide, you must install the following tools on your computer. If you haven't installed them yet, follow the instructions below:

1. .NET SDK

- **What it is:** A set of tools and libraries to build .NET apps.
- **Download here:** <https://dotnet.microsoft.com/download>
- **Installation tip:** Choose the **latest LTS version of the .NET SDK** (e.g., .NET 9 if available). Download the SDK **not just the runtime**.

To verify installation:

```
dotnet --version
```

2. Visual Studio Code (VS Code)

- You should already have this installed. If not, get it from <https://code.visualstudio.com/>

3. C# Extension for VS Code

- In VS Code:
 - Go to the **Extensions** tab (left sidebar or press Ctrl+Shift+X)
 - Search for “**C#**”
 - Install the extension published by **Microsoft**

Step-by-Step: Create Your First Web API

Step 1: Create a New Project

Open your **terminal** in VS Code or your operating system and type:

```
dotnet new webapi -o MyFirstApi
```

```
cd MyFirstApi
```

This command:

- Creates a new folder called MyFirstApi
- Sets up a new Web API project using the **minimal API** style, which doesn't include controllers by default

You'll see files like Program.cs, and possibly `WeatherForecast.cs` and `WeatherForecastController.cs` depending on your .NET version.

Important: The template does not include a `Controllers` folder by default. You'll need to create it.

Step 2: Open the Project in VS Code

```
code .
```

This opens the current folder in VS Code.

If prompted, allow required assets to be added. These help with debugging and building the app.

Step 3: Modify Program.cs to Support Controllers

By default, the minimal API template doesn't support controllers. To enable controller support:

1. Open the Program.cs file.
2. Replace its contents with the following:

```
var builder = WebApplication.CreateBuilder(args);  
  
// Add services to the container.  
  
builder.Services.AddControllers();  
  
var app = builder.Build();  
  
// Configure the HTTP request pipeline.  
  
// Remove HTTPS redirection so you can test with http  
  
// app.UseHttpsRedirection();
```

```
app.UseAuthorization();
```

```
app.MapControllers();
```

```
app.Run();
```

Note: We commented out `app.UseHttpsRedirection();` so you can test your API with `http://` instead of `https://`, which can sometimes be problematic in early development stages.

Step 4: Create the Controllers Folder

1. Right-click the root of the project
2. Click **New Folder**
3. Name it `Controllers`

This is where you define **controllers**. Controllers manage the logic of your app and respond to HTTP requests.

Step 5: Create Your Own Controller

Let's add a controller that returns a list of products.

1. Right-click the Controllers folder
2. Click **New File**
3. Name it `ProductsController.cs`
4. Paste this code into the file:

```
using Microsoft.AspNetCore.Mvc;
```

```
using System.Collections.Generic;
```

```
namespace MyFirstApi.Controllers
```

```
{
```

```
    [ApiController]
```

```
    [Route("api/[controller]")]
```

```
    public class ProductsController : ControllerBase
```

```
    {
```

```
        [HttpGet]
```

```
public ActionResult<List<string>> Get()

{

    return new List<string> { "Apple", "Banana", "Orange" };

}

}
```

What this does:

- Defines a route: api/products
- Handles GET requests to that route
- Returns a list of strings as product names

Step 6: Run Your API

In the terminal, type:

```
dotnet run
```

You should see output like:

```
Now listening on: http://localhost:5000
```

Note: Your port number may be different. For example, instead of "5000" you may see a different number. Use the number that displays in your terminal instead of 5000.

Open a browser and go to:

```
http://localhost:5000/api/products
```

You should see:

```
["Apple", "Banana", "Orange"]
```

Step 7: Understand Routing Basics

- `[HttpGet]` maps a method to an HTTP GET request
- `[Route("api/[controller]")]` means the route is based on the controller name (ProductsController → products)
- You can define custom routes too:

```
[HttpGet("featured")]
```

```
public string GetFeaturedProduct() => "Mango";
```

Now available at:

```
http://localhost:5000/api/products/featured
```

Step 8: Add More HTTP Methods

In your `ProductsController`, you can add methods for `POST`, `PUT`, and `DELETE`:

```
[HttpPost]
```

```
public ActionResult<string> Post([FromBody] string newProduct)
```

```
{
```

```
    return $"Added: {newProduct}";
```

```
}
```

```
[HttpPut("{id}")]
```

```
public ActionResult<string> Put(int id, [FromBody] string updatedProduct)
```

```
{
```

```
    return $"Updated product {id} to: {updatedProduct}";
```

```
}
```

```
[HttpDelete("{id}")]
```

```
public ActionResult<string> Delete(int id)
```

```
{
```

```
    return $"Deleted product with ID: {id}";
```

```
}
```

Use **Postman** to test these methods. You can install Postman from their website and use the free version here: <https://www.postman.com/> 

Step 9: Practice and Build

Now that you have a basic Web API:

- Try returning real data models (not just strings)

- Add validation and error handling
- Store and retrieve data using collections, files, or databases (future lessons will help)

Conclusion

You've now built and tested a simple Web API using ASP.NET Core in VS Code. As you move forward, this setup becomes the foundation for more advanced features like authentication, databases, and real-world deployment